

Grid Sample Operator

Overview

This project implements the Grid Sample operator using Triton. The objective is to reproduce the behavior of PyTorch's `torch.nn.functional.grid_sample` while exploring GPU kernel optimization techniques and evaluating performance against the native PyTorch implementation.

Operator Description

The Grid Sample operator performs spatial sampling on an input tensor according to coordinates provided in a sampling grid. For each output location, the operator retrieves values from neighboring input pixels using bilinear interpolation.

In this implementation, the following configuration is supported:

- 4D input tensors (N, C, H, W)
- Bilinear interpolation
- Zero padding mode
- `align_corners = False`
- CUDA execution through Triton

Implementation Process

Step 1: Environment Setup

The development environment was prepared using PyTorch and Triton. GPU availability and Triton installation were verified before development.

Step 2: Baseline Implementation

The operator behavior was first validated using PyTorch's native `grid_sample` function. This served as the correctness reference throughout development.

Step 3: Triton Kernel Development

A custom Triton kernel was implemented to:

- Read sampling coordinates from the grid tensor
- Convert normalized coordinates into image coordinates
- Identify neighboring pixels
- Perform bilinear interpolation
- Apply zero-padding for out-of-bound coordinates

Step 4: Correctness Verification

The Triton implementation was compared against PyTorch outputs using multiple tensor shapes and test cases.

The maximum absolute error remained approximately within the floating-point tolerance range (10^{-7}), demonstrating numerical correctness.

Step 5: Performance Optimization

Several kernel configurations were evaluated by varying:

- Block size
- Number of warps

The objective was to improve memory access efficiency and GPU utilization.

Benchmark Results

Three benchmark cases were evaluated.

Case 1

Shape:

- $1 \times 32 \times 64 \times 64 \rightarrow 64 \times 64$

Result:

- PyTorch remained slightly faster than Triton.

Case 2

Shape:

- $4 \times 64 \times 128 \times 128 \rightarrow 128 \times 128$

Result:

- Triton achieved approximately 2.03× speedup compared to PyTorch.

Best configuration:

- Block Size = 256
- Num Warps = 8

Case 3

Shape:

- $8 \times 64 \times 256 \times 256 \rightarrow 256 \times 256$

Result:

- Triton achieved approximately $4.83\times$ speedup compared to PyTorch.

Best configuration:

- Block Size = 128
- Num Warps = 4

Key Findings

- The Triton implementation produced outputs consistent with PyTorch.
- Small workloads did not benefit significantly from custom kernel execution.
- Medium and large workloads showed substantial acceleration.
- Kernel tuning had a significant impact on overall performance.

Conclusion

A custom Grid Sample operator was successfully implemented using Triton. Correctness was validated against PyTorch, and performance tuning demonstrated meaningful speedups for larger tensor sizes. The project highlights the effectiveness of Triton for developing optimized GPU kernels while maintaining numerical accuracy.